

Chapters 4 and 6: Functions

EEC 3150

Trevecca Nazarene University

Outline

- 1 Functions
- 2 Scoping
- 3 Functions as Objects and Higher-Order Programming
- 4 Specifications
- 5 Recursion
- 6 Using Functions to Modularize Code

Functions

Motivating functions:

- Consider the program we wrote that used bisection to approximate the square root of 2
- Suppose that we:
 1. wanted to find the root of a different number
 2. wanted a different order root
 3. wanted a different level of precision (ϵ)
 4. were writing a program which needed to evaluate several roots
- All of these cases would require us to copy the code and change the variable values at each occurrence of the code (this is a pain)
- Instead, we can write a function that performs bisection by taking the parameters above (root order, argument of the root, precision) as input and returning the result and then simply call the function whenever we need it

Defn: function

Functions are essentially labeled blocks of code which can be called by using their label instead of manually copying the block of code (similar to how we can set `x = 10` and then use `x` instead of `10`)

Prototypical function definition:

```
def my_function( arguments ):  
    block of code  
    return output
```

(Note: italics in code snippets mean "some code goes here")

- Use the keyword `def` to start the function definition
- Name the function,
- List arguments (0 or more, comma-separated)
- End line with colon
- Indent the body of the function
- return any output

Example: hello function

```
def hello( name ):  
    print('hello, ' + name)
```

- Accepts a string name as input
- Doesn't return anything, but prints to the console

Important points:

- Merely defining the function won't run anything
- It must be called (AFTER it's definition)

Functions

Example: area of triangle

```
def AreaOfTriangle( base, height ):  
    return base*height/2  
# end
```

- Accepts base,height as arguments
- Returns area

Example: sum A to B

```
def sumAtoB( A, B ):  
    total = 0  
    for i in range(A,B+1):  
        total += i  
    # end  
    return total  
# end
```

- Sums integers from A to B
- Accepts A and B as arguments
- $B+1$ in `range()` since it stops one short
- Returns total

Functions

Example: defining several functions in one program

Calculate the diameter, perimeter, and area of a circle given the radius:

```
def diameter( r ):
    return 2*r
# end
```

```
def perimeter( r ):
    pi = 3.1415
    return 2*pi*r
# end
```

```
def area( r ):
    pi = 3.1415
    return pi*r**2
# end
```

- We can define as many functions as we want
- Create a variable for pi

Example: calling a function from a function

Calculate the diameter and perimeter of a circle given the radius:

```
def diameter( r ):  
    return 2*r  
# end  
  
def perimeter( r ):  
    pi = 3.1415  
    d = diameter( r )  
    return pi*d  
# end
```

- We can call diameter from perimeter
- We can even define perimeter before diameter (Python is smart enough to find diameter when interpreting the definition of perimeter)

Example: bisection root

Let's now write our bisection root-finder as a function:

```
1 def find_root(x, power, epsilon):
2
3     # Find interval containing answer
4     if x < 0 and power%2 == 0:
5         print("can't find even root of negative number")
6         return None #Negative number has no even-powered roots
7     # end
8
9     # define values
10    low = min(-1, x)
11    high = max(1, x)
12    ans = (high + low)/2
13
14    # Use bisection search
15    while abs(ans**power - x) >= epsilon:
16        if ans**power < x:
17            low = ans
18        else:
19            high = ans
20        # end
21        ans = (high + low)/2
22    return ans
23 # end
```

- Accepts 3 arguments:
 1. x: the number to be "rooted"
 2. power: the power of the root
 3. epsilon: the error tolerance
- Returns None if it can't compute the root

Positional vs. keyword arguments:

There are 2 ways to provide arguments to Python:

- Positional: listing the arguments in order
- Keyword: listing them in any order but using their names

```
def print_name(first, last):  
    print(first, last)  
# end
```

```
print_name( 'walter', 'white' )  
print_name( first='walter', last='white' )  
print_name( last='white', first='walter' )
```

NOTE: any positional arguments must come BEFORE ALL keyword arguments

Functions

Default values for arguments:

You can set the default value of an argument in the first line of a function definition (this value will be used if you don't supply the argument when calling the function):

```
def print_name(first, last='white'):
    print(first, last)
# end
```

```
print_name( 'walter' )
print_name( 'walter', 'white' )
print_name( first='walter', last='white' )
print_name( last='white', first='walter' )
```

NOTE: any keyword arguments must come AFTER ALL positional arguments

Variable number of arguments:

Like with `max()` function, you can provide a variable number of arguments to a Python function by using the *unpacking operator* (*):

```
def mean( *args ):
    tot = 0
    for a in args:
        tot += a
    # end
    return tot/len(args)
# end
```

- `args` is a *tuple* (a sequence of values, more on that in the next chapter)
- `len()` is a function which returns how many elements are in a tuple (also works for strings)
- Like with strings, you can index individual elements using brackets `[]`: `args[0]` will give the 0th element

Exercise: `max()`:

Let's write our own version of the `max()` function (HINT: `float('-inf')` will be useful)

Multiple returns:

Functions can also return multiple values:

```
def DiamPerim( r ):
```

```
    d = 2*r
```

```
    pi = 3.1415
```

```
    p = pi*d
```

```
    return d, p
```

```
# end
```

```
d, p = DiamPerim( 10 )
```

- Separate returned values with commas

Scoping

Example: scoping

What will this code do?

```
def f():  
    x = 1  
# end  
  
print(x)
```

Scoping

Example: scoping

What will this code do?

```
def f():  
    x = 1  
# end  
  
print(x)
```

- This code will not run
- Since `x` was defined in `f`, it can only be accessed inside `f`
- Even calling `f()` won't make it work
- The *scope* of `x` is only the function `f`

Defn: scoping

The **scope** of an object in Python (or any programming language) the section(s) of a program in which the object is defined and can be accessed

Consequences:

- Variables/functions may not be aailable in some parts of the code
- The same variable name may refer to different objects in different section of code

Example: scoping

What will this code do?

```
def f(x):  
    print(x)  
# end  
  
f(1)  
print(x)
```

- Scoping also applies to the arguments of functions
- `x` can't be accessed outside the function
- Of course, we could define a new `x` variable outside the function, but it would have no relation to the one inside the function

Example: scoping

```
def f():  
    print(x)  
    # x += 1  
# end  
  
x = 1  
f()
```

- There are `x` variables both inside `f()` and the program (module) it's defined within
- Running the program as it is will print 1 (without a separate definition of `x` specifically for `f`, it assumes that the module's value of `x` is what we want)
- However, if we uncomment `x = 2`, then `f` now has its own definition of `x`, which it tried to access (by printing) before it was defined

Defn: global vs. local variables

- **Local variables:** variables with a limited scope (usually limited to a certain function or class)
- **Global variables:** variables with an unlimited scope that can be accessed from ANY function via the `global` keyword

```
def f():  
    global x  
    x += 1  
# end
```

```
x = 1  
f()  
print(x)
```

- The `x` defined outside of `f` is global, but `f` can't modify it
- Normally, any `x` defined within `f` would be local (its scope would be limited to `f`)
- However, using the `global` keyword inside `f` allows the function to change `x`

Example: nested scoping

Let's explore this code:

```
def f():  
    def g():  
        y = 2  
    # end  
  
    x = 1  
# end
```

What it does:

- Defines a function `f`
- Objects whose scope is `f`: `x`, `g()`
- Objects whose scope is `g`: `x`, `y`

Let's try:

- Calling `f()`
- Calling `g()` in `f()`
- Printing values of variables to test scoping
- Define `z` outside both functions

Defn: call stack and frame

- A **call stack** is a data structure which stores information about currently active functions within a program
- A **frame** is an item in the call stack which refers to a particular function
- We say that frames are *pushed onto* and *popped off* of the stack

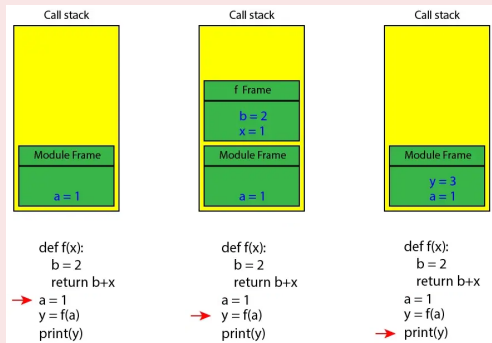


Figure: Source: towardsdatascience.com

Call stack and errors:

One place where understanding how a call stack works in when debugging errors:

```
def f():  
    return 1+'c'  
# end  
f()
```

- This code will throw an error because we can't add an integer and a string
- The error message will acknowledge that the error occurred in `f()`

Call stack and errors

One place where understanding how a call stack works in when debugging errors:

```
def f():  
    def g()  
        return 1+'c'  
    # end  
    g()  
# end  
f()
```

- This code will throw an error because we can't add an integer and a string
- The error message will acknowledge that the error occurred in g() INSIDE f()

Functions as Objects and Higher-Order Programming

Functions as Objects:

```
def abs2(x, F, G):  
    if x > 0:  
        return F(x)  
    else:  
        return G(x)  
  
def f(x):  
    return x  
  
def g(x):  
    return -x
```

- Functions are objects just like int's and float's
- Try `type(abs)`
- Therefore, you can pass/return functions to/from other functions
- This is called *higher-order programming*

Recursion

Defn: factorial

The factorial function takes an integer and multiplies all positive integers up to (and including) itself:

$$n! = n(n-1)(n-2) \cdots 3 \cdot 2 \cdot 1$$

It can be more properly defined in an inductive/recursive fashion using the following recurrence relation:

$$1! = 1$$

$$n! = n \cdot (n-1)!$$

This second definition gives us a different perspective on how it operates (and, thus, how to code it)

Two ways to write a factorial function:

```
def fact1(n):  
    ans = 1  
    for i in range(1,n+1):  
        ans *= i  
    return ans
```

```
def fact2(n):  
    if n == 1:  
        return 1  
    else:  
        return n*fact2(n-1)
```

- The first function simply uses a for loop to loop through all the relevant integers (in the style of the first definition of factorial)
- The second function actually calls itself with an argument one less than the previous call to compute factorial (in the style of the second definition)
- The second function is an example of *recursion*

Defn: recursion

In general, **recursion** is a technique for solving a problem which breaking into simpler version of the same problem

- In a programming context, this consists of making a function call itself in such a way that after each recursive call, the problem gets simpler
- Ex: each time `fact2()` calls itself, the argument is one less

Parts of a recursive function (you can have one or more of each):

- Recursion case: where the function calls a simpler version of itself
- Base case: the simplest possible case where recursion finally ends

Without a base case, there will be “infinite” recursion (rather recursion until the call stack gets full and you run out of memory)

Recursive factorial cases:

```
def fact2(n):  
    if n == 1:  
        return 1  
    else:  
        return n*fact2(n-1)
```

- Base case:
return 1
- Recursion case:
return n*fact2(n-1)

Fibonacci sequence:

The Fibonacci sequence $F(n)$ is defined recursively as follows:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2)$$

Write a recursive and a non-recursive Fibonacci function

Using Functions to Modularize Code

Modularizing with Functions

Defn: modularization

Modularization is the process of writing large programs using many smaller sub-programs (aka, functions), thus making the code more organized and readable

Modularizing the hyperbolic trig functions:

Here are the hyperbolic trig functions:

$$\sinh(x) = \frac{e^x - e^{-x}}{2}$$

$$\cosh(x) = \frac{e^x + e^{-x}}{2}$$

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Write several Python functions:

- an exponential function
- a function for each trig function
- a function which returns all trig functions

NOTE: each function will use the one before it

Specifications

`help()`:

Run these commands in the console:

```
help(abs)
help(max)
help(len)
help(print)
```

- The `help()` function provides info about functions (and other objects)
- Prints the name, arguments, and description of a function
- The descriptions come from something called a *docstring* (let's make our own example)

Docstring:

Define this function and run `help()` on it:

```
def f(x):  
    '''  
    this is a docstring  
    '''  
    return x+1  
# end  
  
help(f)
```

- Triple quotes define a multi-line comment (single or double)
- When placed at the beginning of a function definition, they make something called a **docstring**
- Docstrings are used to describe the purpose of the function (see the previous examples)
- i.e., docstrings provide the **specifications** of a function

Example: docstring

```
def mean( *args ):
    '''
    This function returns
    the mean of the
    variable number of
    arguments (float or int)
    provided
    '''
    tot = 0
    for a in args:
        tot += a
    # end
    return tot/len(args)
# end
```

- Here is our mean code with a docstring providing its specification